

Raseen2 Raseen2

RASEEN2

-  Quick Submit
-  Quick Submit
-  Presidency University

Document Details

Submission ID**trn:oid:::1:3553156142****Submission Date****Apr 28, 2026, 9:38 AM GMT+5:30****Download Date****Apr 28, 2026, 9:43 AM GMT+5:30****File Name****Research_paper-ccs0020.pdf****File Size****263.4 KB****9 Pages****3,563 Words****20,739 Characters**

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Transformer based end-to-end Web Application Firewall (WAF) Pipeline

Dama Karthikeya
School of Computer
Science & Engineering
Presidency University
Bengaluru, India
DAMA.20221ccs0020@
presidecyuniversity.in

Revanth Maski
School of Computer
Science & Engineering
Presidency University
Bengaluru, India
revanth.20211ccs0151
@presidecyuniversity.in

Shaik Mohammed Ejaaz
School of Computer
Science & Engineering
Presidency University
Bengaluru, India
SHAIK.20211CCS0024
@presidecyuniversity.in

MS. Rakheeba Taseen
School of Computer Science
& Engineering
Presidency University
Bengaluru, India

Abstract - Growing adoption of web techniques have led to rising insecurity to malicious actors attacking web sites, while existing solution are no longer flexible enough to counter this constant change. This paper presents a novel system to construct and evaluate a new Transformer-based, end-to-end WAF pipeline, that leverages learned semantic representation of HTTP requests instead of traditional signature-matching algorithms. Employing Distiller, a pre-trained neural language model adapted with a fine-tuning step on a carefully selected corpus of HTTP messages, our method makes it possible to identify potentially harmful requests, that may slip through binarily rule-based legacy systems due to obfuscation or contextual novelty.

Our experiments achieve a macro F1-score of 0.94 compared to only 0.70 for a ModSecurity/OWASP CRS reference model.

Our pipeline is able to detect attacks in the form of SQL injections, cross-site scripting, and command injections at a high rate of 95%, while keeping the error rate below 0.1%. Compared to our baseline, our detection rate is improved by at least 35% when considering missed cases. With respect to performance characteristics, the pipeline can process up to 800-1,200 requests per second with latencies ranging from 15 to 25 ms, which is sufficient for production usage.

I. Introduction

Web applications have become a critical part of infrastructure used to carry out business transactions, government work, health care, and personal communications. Their proliferation, although welcome, makes them easy targets for attack.

While the nature of web application attack has

evolved over time, SQL Injection and Cross-Site Scripting are still heavily used by hackers today, as they seek to leverage new zero-day, context-sensitive toolkits designed to defeat automation tools [1].

Consequently, safeguarding web applications has become a key concern in IT and computer science. In the past, the only web application security technique widely employed was having Web Application Firewall solutions in place, which function by comparing the request against a set of attack signatures (pattern matching). While this solution was fairly useful, it is intrinsically limited in the extent of security it provides.

In our deployment of WAF mechanisms, the current problems we encountered include: Zero day attack. Any attack runs through the rules, it can monitor only one attack it was designed for. Any other attack can bypass the rules, which do nothing to detect that attack. Payload obfuscation. Encoding as URL percent encoding, gzip encoding and may make the malicious payload a little difficult to determine.

II. Related Work

The signature-based architecture of WAFs has been prevalent in business applications for more than a decade now. An excellent case study would be the ModSecurity web application firewall used in combination with the Open Web Application Security Project (OWASP) core rule set. Signature-based WAFs work flawlessly against known patterns; however, what limits their capability is the rule set that they use. Attackers are already familiar with this, and they often alter their attacks to escape detection, either through URL encoding, Base64 obfuscation, or adding comment tokens to SQL strings. Even more difficult is the zero-day scenario, where

signatures cannot work because there must be a rule before the signature can become Signature based architecture of WAFs has been in use for larger business applications for more than 10 years now. An excellent example will be to see the ModSecurity web application firewall on top of the Open Web Application Security Project (OWASP) core rule set [2]. Signature WAFs work well against known protocol names and their embodiment but the limitation lies in the rule set that they use. Attackers already know that so they keep modifying the attack to escape detection, adding URL encoding, Base64 encoding, or comment tokens to the SQL strings. The trickier ones are the zero day attacks, as signatures will be useless here as a rule has to exist before signature can be enabled [3]. The motivation for using machine learning in such contexts also stemmed from the above limiting factors. Early work used the Support Vector Machine (SVM), the Random Forests (RF), and the k-Nearest Neighbors (kNN) algorithms for attack type classification and anomaly detection tasks [4]. It was promising as trained models could learn more by providing more details to its learning process but there were some issues. Deriving features from raw HTTP traffic data was not an easy task; domain knowledge is very costly and even crafted features are sometimes not able to differentiate between subtle variations between two fields.

III. Methodology

We realized our implementation of WAF as a series of loosely coupled services; traffic hits the edge ingestion layer, undergoes preprocessing to create inputs from the model, is run through the Distil BERT inference engine and eventually arrives at the decision and enforcement modules.

Having each of these pipeline stages implemented as a separate service made it easier to design, test and scale.

- **Edge Ingestion Layer:** An L7 proxy such as Nginx or Envoy directs all HTTP requests to a WAF component that processes it later. This module is also responsible for load balancing and traffic scheduling among all WAF instances.
- **Data Preprocessing Module:** Each request is decoded normalized serialized and transformed into sequences of tokens, which can be examined with the Transformer algorithm.
- It was apparent, that preprocessing of the request was equally essential as the machine learning model, when it comes to identifying malicious traffic.
- **Transformer inference engine:** FastApi server, hosting a fine-tuned distil BERT model, which performs real-time classification of sequences from the previous module.
- **Decision Engine:** accepts the confidence values from the model and makes decisions to allow, challenge or block the specific request as per the configurable security policy. Further capabilities include plugging in threat intelligence sources from other third party data providers.
- **Enforcement Module:** decides according to decision engine, refuse malicious request or prevent it from arriving to frontend server while passing good request..

IV. Module-wise Implementation Details

One of the module in the pipeline which merits more attention is the Preprocessing module, since its design has a crucial impact in the accuracy of the output. The raw data, read in through HTTP request, can come with a wide variety of formats some fields are multi-encoded, clients treat space characters differently and highly entropy generated tokens (e.g. sessions IDs) bring an enormous amount of flexibility for the model to digest.

The Data Preparation and Encoding Module has been deployed as a light sidecar service which can be easily plugged in the edge intake layer or operate independently depending on the circumstances. The text below highlights its main functionalities: **Parsing of HTTP Requests:** httptools is a python package that safely breaks the raw incoming HTTP requests into these components (method path query string, header and payload).

It dissects the message and generates the individual sections to be addressed later.

URL Decoding: urllib.parse.unquote applies decoding on percent-encoded characters in path, query, and form body fields.

Base64 Decoding: Strings conforming to Base64 format are identified and decoded, specifically those used in request bodies and headers as an encoding technique.

HTML Entity Decoding: Character references (e.g., < and ') are converted to characters prior to processing by the model.

Normalization: A consistent set of normalization steps are performed to minimize noise:

Lowercase all text, ensuring casing doesn't impact the analysis.

Whitespace sequences and new lines are normalized to single spaces.

High entropy non-discriminative tokens (UUIDs, session IDs, IP addresses) are substituted with categorical tags ([UUID], [IP_ADDR]), to avoid having the model learn particular values not indicative of attacks.

A quick side note about normalization: the order in which the decoding steps are applied is important. From experimentation we discovered that performing URL decoding before Base64 decoding yielded more consistent results compared to the opposite order. After thorough testing against various samples of encoded payloads we arrived at URL → Base64 → HTML entity decoding as our standard pipeline.

V. Model Architecture and Model Training

DistilBERT was selected as a backbone for classification as it is more feasible. It provides on common benchmark tests approximately 97% performance of BERT at an order of magnitude 60% faster at execution speed, in combination with a 40% reduction in the number of parameters [7].

For a model that is required to perform decision-making judgments on a per request basis under timing constraints, efficiency was more significant than task language performance. An implementation with a full trained BERT model

in same settings would probably provide little advantage on performance and increased latency.

The process started with pre-trained DistilBERT models obtained from the internet and proceeded in the following way:

- **Pre-trained Initialization:** The model was initialized with pre-trained weights of the DistilBERT, which encode language representations trained on general purpose text. Those language representations served as a form of bias, as token co-occurrences in the context of HTTP requests show certain similarities with language structures, despite a different vocabulary.
- **Classification Head:** A linear classification head was applied to the encoder's representation of the [CLS] token of the input sequence. That is, the encoder's context-sensitive representation was mapped to one of five target classes: SQLI, XSS, CI, PT or benign.
- **Loss Function:** Multi-class CCE (categorical cross-entropy) loss function.
- **Optimizer & Schedule:**
Adam optimization, with linear warm-up and learning rate decay. Experimentally, the linear learning rate schedule proved to be more stable than keeping the learning rate at a constant value, especially during the first several epochs of training.
- **Batch Size and Early Stopping:** Batch size between 16 and 32. Between 3 and 5 epochs of training, with early stopping based on validation loss reaching a plateau. Attack patterns are highly predictable in nature; thus, it was crucial to use early stopping to avoid overfitting.

- **Hardware:** Train using NVIDIA V100 or A100 GPUs with PyTorch + CUDA, training times were reasonable and allowed for multiple parameter sweeps.
- **Monitoring:** Logging training loss and accuracy, validation loss and accuracy, and additional metrics like precision, recall, and F1 score. It is important to monitor a variety of metrics to detect any imbalances that accuracy might miss.

VI. Algorithms

It is the transformer encoder module used by DistilBERT which allows it to have relational reasoning abilities, which are not naturally present in any rule-based system. While a rule-based system would simply compare individual tokens based on predefined patterns, the transformer encoder creates representations of tokens in terms of their relevance in relation to the entire string input. In terms of HTTP request analysis, this is relevant because the token SELECT is significant in the presence of another token, namely FROM.

The principal architectural elements are:

- **Self-Attention Mechanism:** Each token in the sequence is associated with three vectors called Query (Q), Key (K), and Value (V). Attention scores are obtained by calculating the dot products between Q and K after scaling, followed by applying softmax for normalization. The attention scores are then used to weight the values in V, thereby generating context-based representations for tokens. This helps the WAF identify instances where disparate request parts join together to create an attack pattern.

- **Multi-Head Attention:** The attention mechanism is computed in parallel for multiple heads, where each head pays attention to its own linear transformation of the input sequence. For instance, in our experiment, visualizing learned attention weights revealed that each head had its own tendency to pay attention to certain components of the HTTP format (HTTP methods/paths or SQL/JavaScript tokens).
- **Position-wise-Forward-Networks:** After each attention block, there follows a two-level feed-forward network that operates separately on each position. This provides more depth to the representations than attention alone.
- **Residual Connections and Layer Normalization:** Residual Connections and Layer Normalization: Residual connections and layer normalization surround each sub-layer. The addition of these modules was essential for our smaller dataset during fine-tuning, as the dataset was small and specific to our task.

VII. Integration of Components

The Web Application Firewall (WAF) has been developed as a microservices architecture, where each microservice is isolated in its own Docker container. We proposed the WAF is fully built with microservices to enhance separate testing, updating, and scalability (e.g. the horizontal scalability of the inference engine may be implemented independently without the preprocessor).

Docker containers: Each microservice (edge layer configuration, pre-processor, inference engine, decision engine, monitoring dashboard)

runs on its own container which was beneficial in supporting the development of WAF and also in providing a standard deployment in all environment and less extra effort to onboard new comers. This strategy proved helpful in facilitating the process of developing the WAF and ensured standard deployment across various environments, reducing the effort required for onboarding new members of the team.

Fast API communication between services. The preprocessor communicates with the inference engine using REST APIs. The asynchronous approach in Fast API became a vital factor in maintaining low latency in response to simultaneous request traffic. Redis caching: common patterns and identifiers are cached to minimize the need for repeated inference queries during high traffic from repeat offenders.

Influx DB all security events, security decisions made by the decision engine, and latency statistics are sent to Influx DB, providing a clean database based on a timeseries format for the security dashboard.

Nginx/Envoy configuration the edge layer directs the incoming request to the preprocessor, executes security decisions made by the decision engine by forwarding the request to the backend or by blocking the request and sending a message back to the user. Lua scripts were written for configuring the Nginx/Envoy to support complex routing logic.

VIII. HANDLING OBFUSCATION AND ZERO-DAY VARIANTS

In fact, obfuscation resilience proved to be the critical factor that distinguished our system from Mod Security. While this characteristic has been previously well documented as a structural weakness of signature-based WAFs, the degree

of the difference was bigger than initially anticipated, especially when it came to multi-layer encoded attacks.

Below are the types of obfuscations used in our test dataset:

- **Encoding Variants:** single/double URL encoding, Base64 wrapping, Unicode escaping, HTML character references.
- **Polymorphic Payloads:** attacks with varying syntax/structure but with a consistent meaning of attack.
- **Fragmentation:** attack payloads split across multiple input variables or HTTP headers with application-based reassembly.
- **Comment Injection:** SQL/JavaScript comments used as part of attack vectors.

The detection rate of the attacks by ModSecurity showed a significant drop for the presented variants. Simple SQL injections, such as UNION SELECT user, password FROM users, did not trigger any alert if the input was doubly encoded or the special non-standard Unicode characters were used in place of their ASCII counterparts. This happened due to the inability of the regular expressions that power ModSecurity's rules to detect them in those conditions.

On the contrary, our model performed noticeably better with regard to dealing with similar attacks. There are three main reasons behind this observation. First, our preprocessing pipeline employs several layers of decoding before passing the normalized input to the model, removing a good part of obfuscation layers along the way. Second, using DistilBERT with its self-attention feature means that semantic

representations, and not mere character sequences, are used as inputs by the model. Thus, the syntax manipulations that would not change semantic structure of the payload would become increasingly difficult. Third, the request encoding format passed to the model captures the relations between multiple fields of the request at once, allowing for an overview that cannot be achieved by per-field rule matching.

A practical example will illustrate how this works. A Base64-encoded version of the SQL injection attack described above was not detected by any of Mod Security's rules; however, our model successfully classified it as malicious. After being decoded by our preprocessing module, Base64 code became visible to the model in the exact same fashion as the original payload was present during training.

While generalization to zero-day-like attack variants is hard to prove rigorously, we do not claim anything more than that. We tested the model on the artificial attack samples built to resemble some attack classes structurally, but not literally. This resulted in noticeably improved performance compared to the signature-based method, even though it was not perfect in terms of detecting all variants..

IX. COMPARATIVE RESULTS

The two systems were implemented in the same test environment, protecting the same vulnerable websites (OWASP Juice Shop and DVWA). It was important for both WAFs to be implemented in the same environment because any difference found in the metrics was due to the difference

between the two WAF implementations and not because of the test environment itself.

The test traffic comprised four categories:

- Benign traffic: User simulation against the target applications..
- Known malicious traffic: Samples selected from the CSIC 2010 and CICIDS2017 benchmark datasets, augmented by custom crafted attacks related to SQL Injection, XSS, Command Injection, and Path Traversal.
- Obfuscated variants: Attacks based on known techniques and modified using the aforementioned encoding and evasion approaches as described in Section VIII.
- Synthetic zero-day-like attacks: Zero-day-like attacks synthetically created to mimic semantically and structurally known attacks but cannot be captured by current WAFs using pattern-based approaches.

Performance measurements included detection rate (true positive rate), false positive rate, false negative rate, request processing latency, and CPU/Memory overhead under load conditions. Detection accuracy metrics were quantified by the macro-F1 score for each WAF, yielding 0.94 for the transformer model, compared with 0.70 for ModSecurity. The percentage of missed attacks was reduced by over 35% in addition to maintaining a low false positive rate of less than 0.1%. Within the test throughput range (800 to 1,200 requests per second), the average request latency varied between 15 and 25 milliseconds, which can be considered a reasonable value for deployment in a real-world scenario.

X. CONCLUSION

The paper describes a pipeline for WAF that includes classification done using a transformer classifier based on the DistilBERT model. The reason behind choosing this classifier is obvious signature-based solutions will not detect obfuscated and semantic new attacks. DistilBERT can be used to recognize attacks through analyzing the semantics of the HTTP request.

The principal findings are summarized as follows:

- **Detection Accuracy:** Achieved macro F1-score was equal to 0.94 compared favorably to a score of 0.70 reported by ModSecurity using OWASP CRS across SQL Injection, XSS, Path Traversal, and Command Injection attacks.
- **Obfuscation Resilience:** Multi-layer preprocessing and semantics-based classification provided significantly improved detection rates for obfuscated or mutated payloads, rendering the baseline rule-based approach ineffective against those payloads.
- **Novel Threat Generalization:** The model demonstrated some ability to detect novel attack variants not included in the training set provided they were similar in structure or semantics to known attacks.
- **False Positive Control:**
Despite heightened detection rates, false positives still constituted less than 0.1% of requests, which is of great practical importance because high false positives might undermine faith in deployed WAFs and lead to less strict rules.

- **Throughput and Latency:** The system was able to process 800-1,200 requests per second (RPS) with each taking between 15 and 25 ms, which is sufficient for most applications. More stringent latency requirements require either hardware-accelerated inference or reduced complexity.
- **Explainability:** Attention weights helped provide insights into how various parts of a request contributed to specific classifications, allowing for more convenient troubleshooting and analysis of flagged requests by security analysts.

Among potential areas for improvement, one should note limitations associated with benchmarking using publicly available datasets which may not faithfully represent production-like workloads. As attackers' techniques develop further, the performance of a model deteriorates unless periodic training takes place. While the latency range of 15-25 ms is appropriate for current test settings, it does not guarantee consistent results in extreme concurrency scenarios without additional infrastructure optimizations.

Directions for future work may involve introducing online continual learning to allow for gradual adaptation of the model based on new attacks without complete retraining. Additionally, researchers could try to implement federated learning which will allow collaboration among organizations for developing a common model based on their traffic without the need for direct sharing of raw datasets. Another avenue for research concerns multimodal detection involving network-level features.

XI. References

- [1] B. R. Dawadi, S. K. Shrestha, and R. Shakya, “Deep Learning Technique-Enabled Web Application Firewall for Attack Detection,” *Sensors*, vol. 23, no. 4, pp. 2073–2091, Feb. 2023, doi: 10.3390/s23042091.
- [2] Z. Long, Y. Zhang, J. Wang, and H. Chen, “A Transformer-Based Network Intrusion Detection Approach for Cloud Security,” *Journal of Cloud Computing*, vol. 12, no. 1, pp. 1–18, 2024, doi: 10.1186/s13677-023-00574-9.
- [3] Y. Yang, Z. Liu, and J. Lin, “BERT-Based Deep Learning Model for Web Attack Detection in HTTP Traffic,” *IEEE Access*, vol. 11, pp. 11823118245, 2023, doi:10.1109/ACCESS.2023.3320197.
- [4] N. Montes, A. C. Silva, and R. Pérez, “Web Application Attack Detection Using Transformer-Based Language Models,” in *Proc. Int. Conf. on Cyber Security and Protection of Digital Services*, 2022, pp. 1–8.
- [5] OWASP Foundation, “Machine Learning in Web Application Firewalls: Next-Generation WAF Architecture,” OWASP Technical Report, 2024.
- [6] H. Athawale, “Building a Transformer based end-to-end Web Application Firewall (WAF) pipeline,” *Medium*, Mar. 03, 2026. [Online]. Available: <https://harshdeepathawale.medium.com/building-a-transformer-based-end-to-end-web-application-firewall-waf-pipeline-704b3d191039>